Local AI with Ollama — Lab Report

Running and developing local AI applications

Setup

Ollama 0.12.6 on Linux with one NVIDIA GPU (32 GB). Models: `qwen3:8b` (Model 1), `llama3.2:3b` (Model 2), `qwen2.5vl:7b` (vision), `nomic-embed-text` (embeddings); all run 100 % on the GPU. The suggested qwen3.5/gemma3 builds require a newer Ollama than the one installed, so the closest supported models were used. Full code and outputs: `local_ai_ollama_lab.ipynb`.

Screenshots

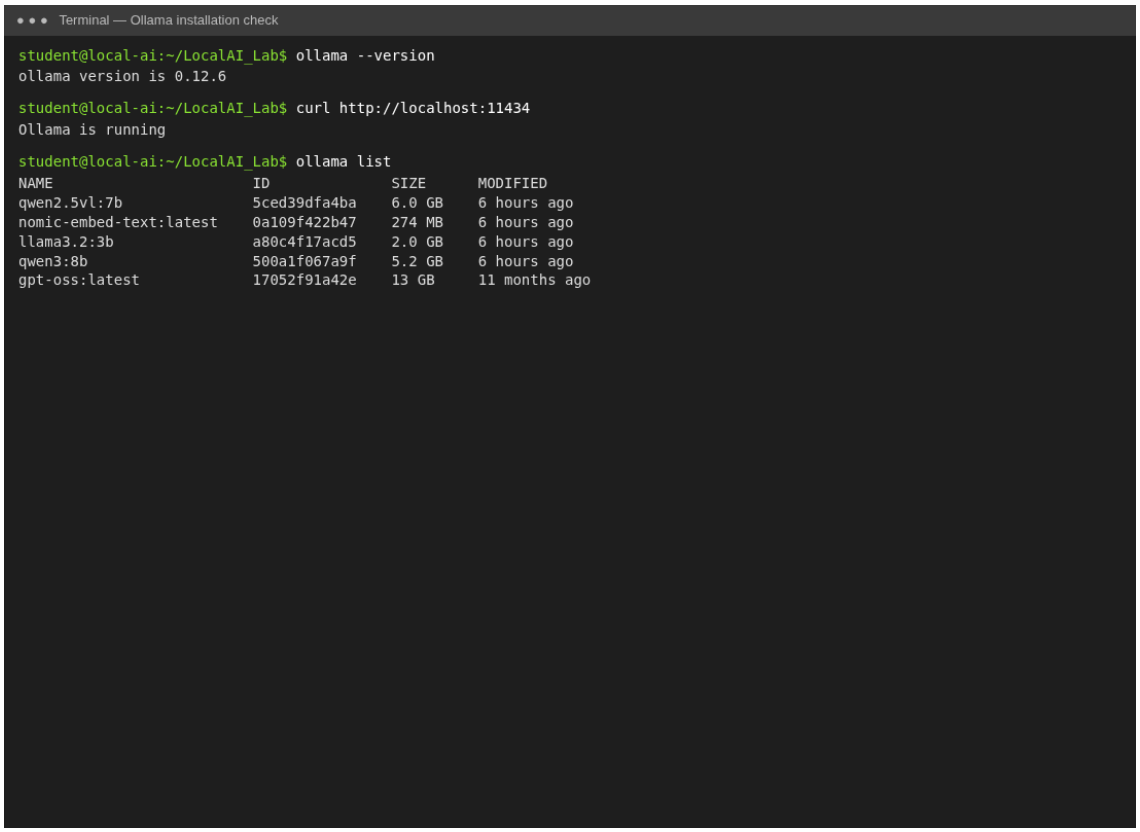


Figure 1: Installation check

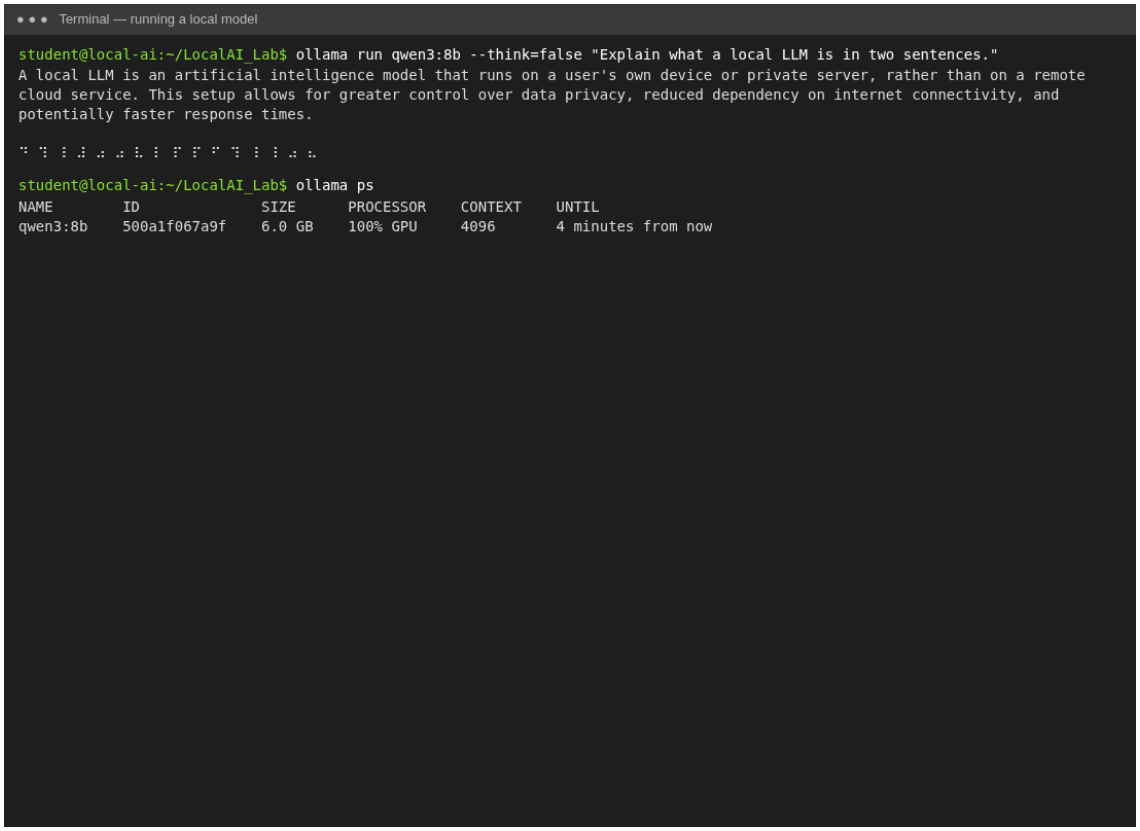


Figure 2: Running a local model

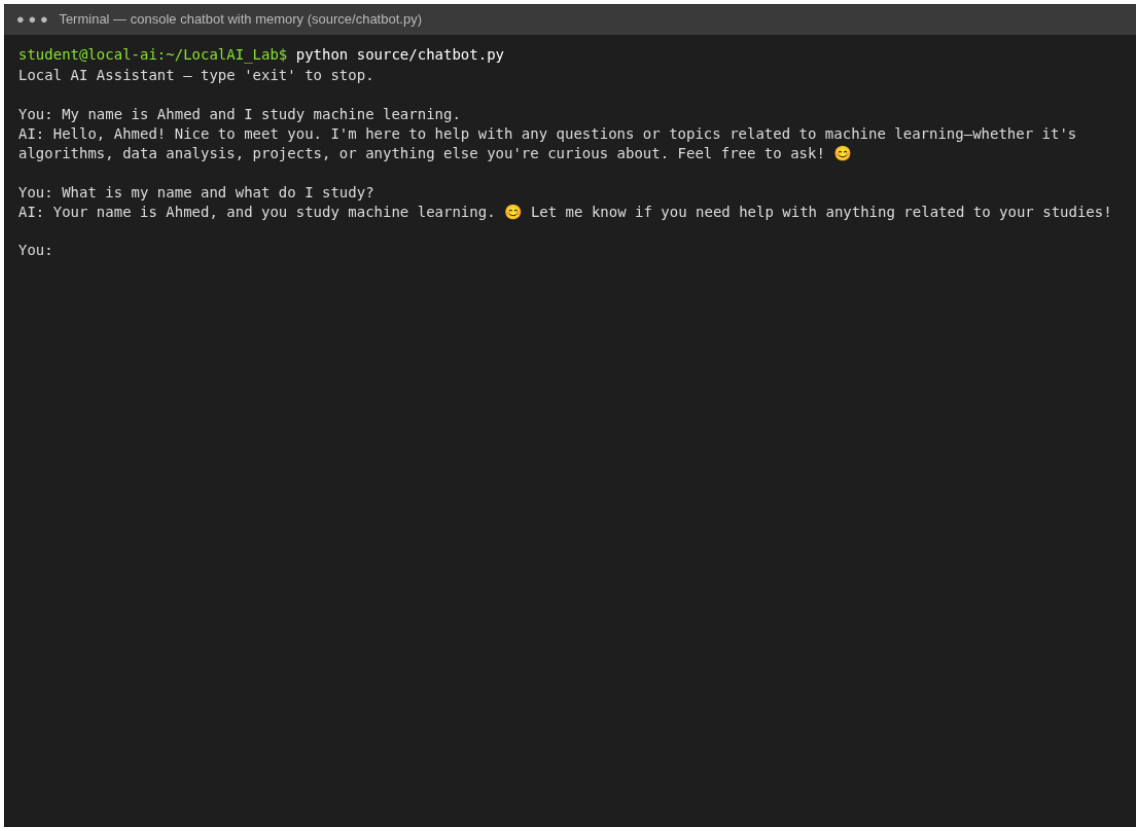


Figure 3: Console chatbot with memory

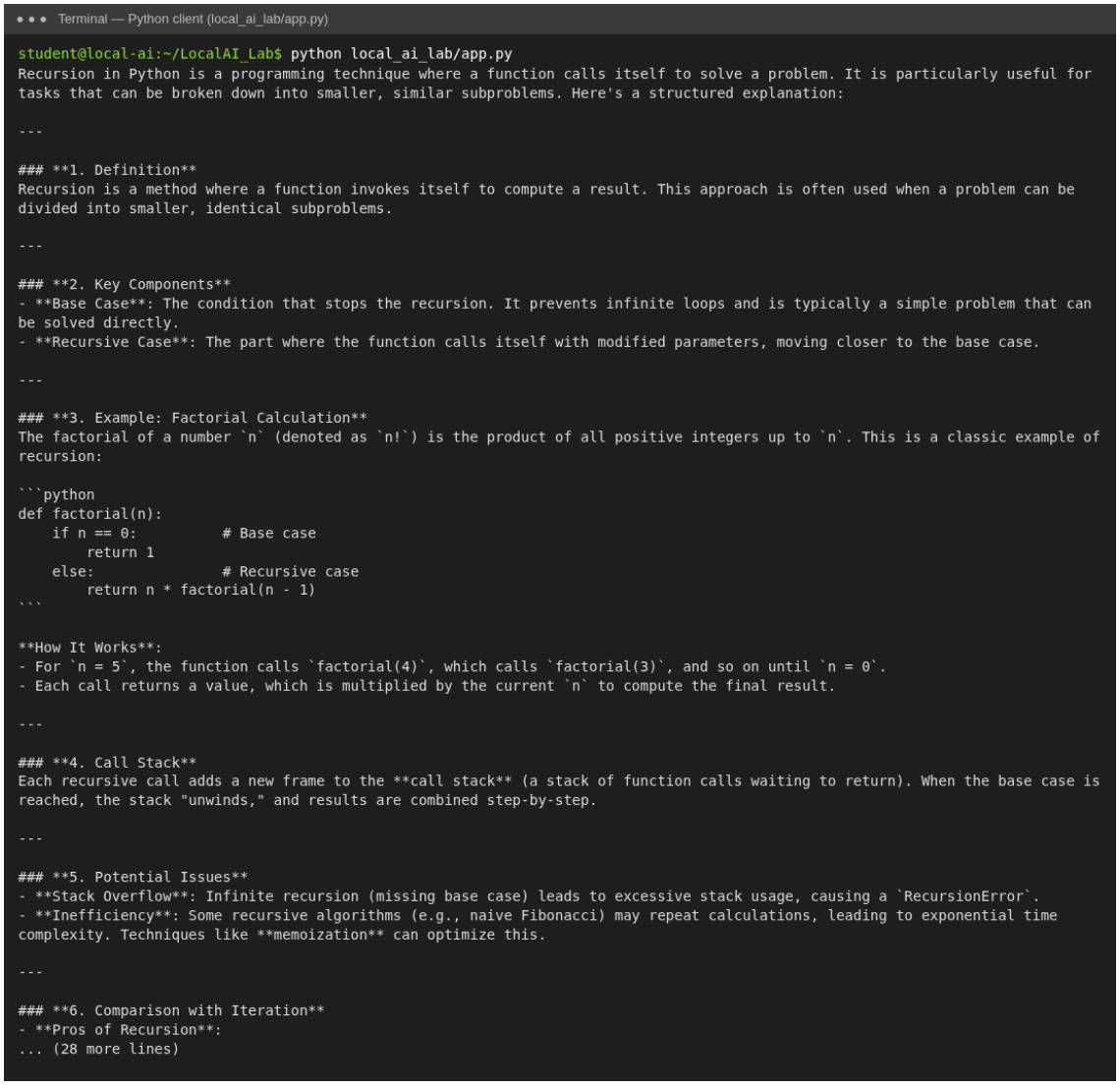


Figure 4: Python client

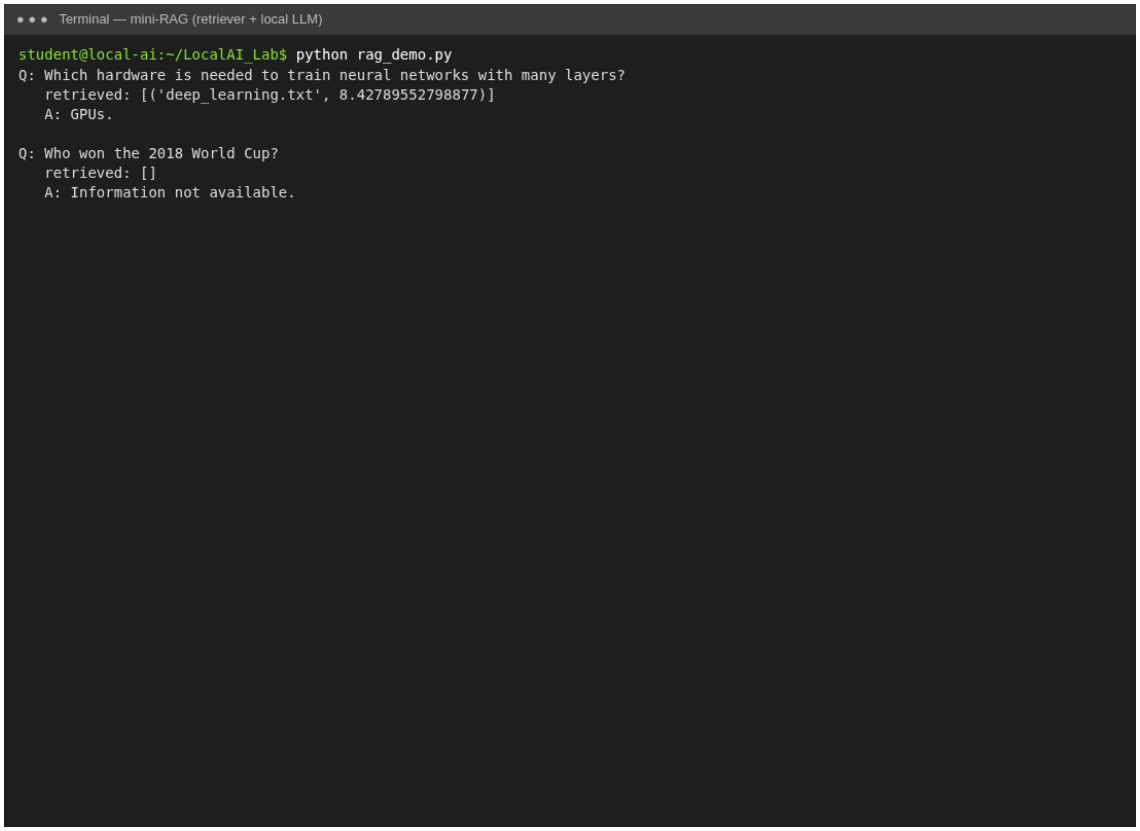


Figure 5: Mini-RAG

Exercise answers

Exercise 3–4: first local LLM

1. **Yes.** All three prompts ask about the same topic, but each one sets a different **audience**, so the model changes its length, vocabulary and structure. The 5-sentence prompt gives exactly five plain sentences in ~3 s. The child version uses analogies ("a super-smart friend", games, Siri) and an emoji. The professor version is a long, structured survey (subfields, bias-variance, P vs NP, AGI) and takes ~10 s. The output is conditioned on the whole prompt, and sampling adds some randomness too (here fixed with seed=42 for reproducibility).
2. **No.** The notebook only talks to `http://localhost:11434`. The `ss` check shows that the Ollama server listens on **127.0.0.1:11434** (loopback only), and no cloud API key is used anywhere.
3. **On this machine.** The Ollama server loads the model weights from the local disk (`ollama list`: qwen3:8b = 5.2 GB, Q4_K_M), and `ollama ps` shows **`100% GPU`**: the forward passes run on the local NVIDIA GPU.
4. **Generation keeps working.** Inference only needs the local weights and the local GPU, and the request never leaves the loopback interface. What stops working without Internet is **downloading** new models (`ollama pull`) and anything that fetches external data. *(The offline test itself — unplugging the network and re-running the cell — has to be done manually on the machine.)*

Exercise 5: prompt engineering

Both prompts give the same **role** ("university professor"), but everything else differs:

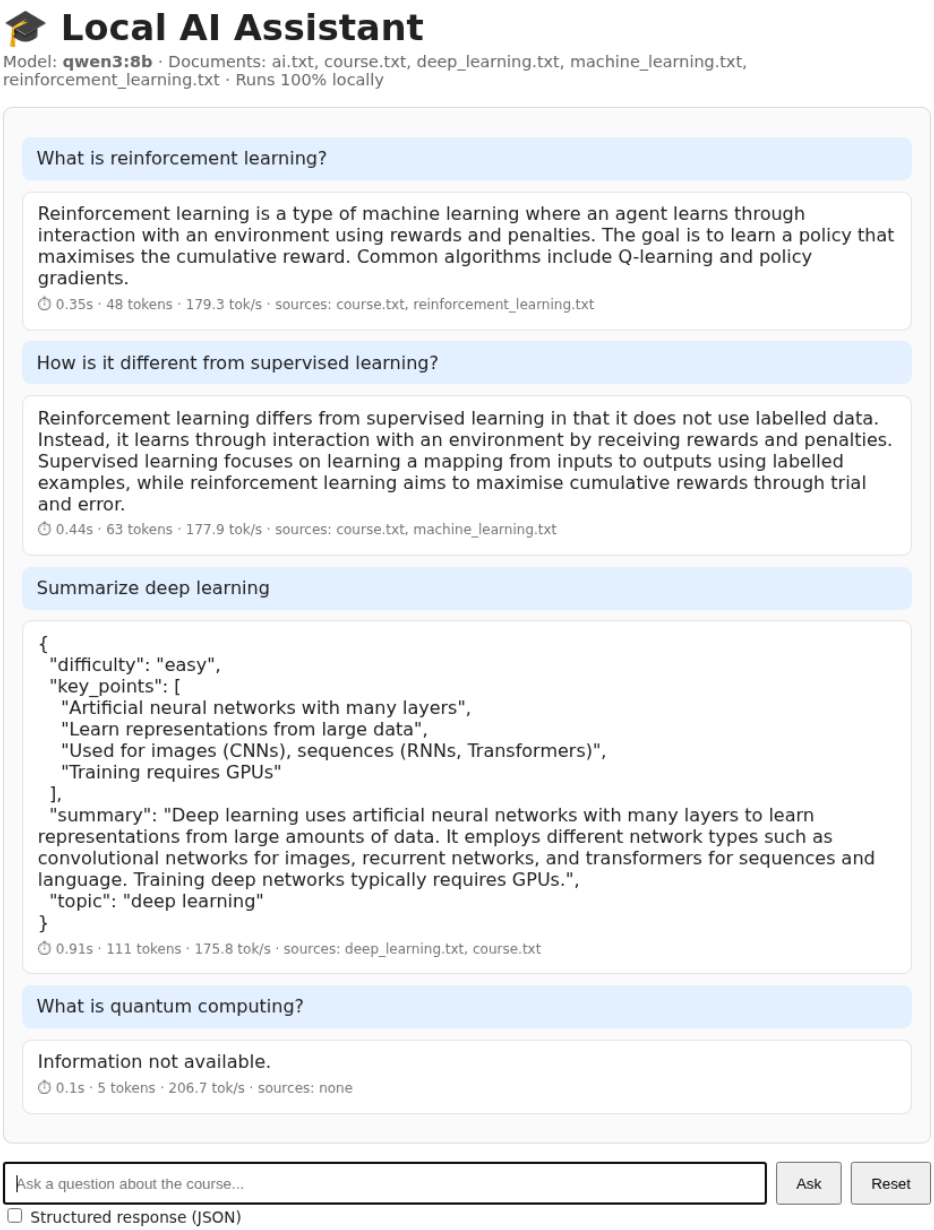


Figure 6: Local AI Assistant web app

Element	Prompt A	Prompt B	Effect observed
Role	professor	professor	Academic, explanatory tone in both answers
Context / audience	none (implicitly experts)	“first-year CS student”	B uses everyday language and a house-building analogy; A uses formal definitions
Constraints	5 mandatory items	“simple language”, “one concrete example”	A covers all 5 items; B keeps to one example and no maths
Output format	numbered list of sections	free	A comes back with 5 matching headings, a matrix table and a code block; B is a short tutorial
Specificity	high (adjacency matrix, Python)	low	A contains much more technical content — but also a mistake: its Python example calls `dag.is_directed_acyclic_graph(` and `dag.topological_sort()`, which are **not** `DiGraph` methods (the real API is `nx.is_directed_acyclic_graph(d` so it would crash

Conclusion: the more precise the prompt (role + audience + constraints + format), the more predictable and checkable the answer. A specific prompt does not guarantee correctness, though: generated code still has to be run and checked.

Exercise 6: model management

- **`ollama list`** (`/api/tags`) shows the models **stored on disk**: name, ID, file size, modification date — plus family, parameter count and quantization through the API (e.g. qwen3:8b: 8.2B params, Q4_K_M, 5.23 GB).
- **`ollama ps`** (`/api/ps`) shows the models **loaded in memory right now**: their size in memory (6.0 GB for qwen3:8b, larger than the 5.2 GB file because of the KV cache and runtime buffers), **where** they run (`100% GPU`), the context length (4096) and **when** they will be unloaded (`UNTIL`, 5 min of inactivity by default).
- A **downloaded** model is just a file in Ollama’s blob store: it costs disk space, no RAM/VRAM, and cannot answer yet. A **loaded** model has its weights copied into GPU/CPU memory and is ready to answer immediately. The first request to an unloaded model pays the **load time** (measured in Part 16: 1.4 s for qwen3:8b, 1.0 s for llama3.2:3b).

Exercise 7: comparing two models

	qwen3:8b	llama3.2:3b
Time	18.7 s (3,065 tokens incl. ~"thinking")	**2.1 s** (192 tokens)
Requirements met	all 5 (type hints, docstring, explanation, complexity O(√n), examples)	type hints + docstring only; no algorithm explanation, **no complexity**
Automatic test (-5…1999)	correct everywhere	**wrong for n = 5**: the shortcut `n % 10 == 5` rejects 5, which is prime
Memory (Part 16)	6.0 GB VRAM	**3.3 GB** VRAM

- **Faster:** llama3.2:3b, both in latency (2.1 s vs 18.7 s here) and in raw speed (≈286 vs ≈173 tokens/s in Part 16). Most of qwen3's time is spent "thinking" (hidden reasoning tokens) before answering.
- **Better code:** qwen3:8b — complete, documented and correct. llama3.2's code looks plausible but has a real bug that only testing reveals.
- **More memory:** qwen3:8b (8.2 B parameters vs 3.2 B).
- **For programming assistance** I would choose **qwen3:8b**: a wrong answer costs much more than 15 extra seconds.
- **The largest model is not automatically the best:** it depends on the task and the hardware. For short, simple answers (chat, summaries, classification) the 3B model is faster and lighter and is good enough; on a machine with little memory it may be the only option. Size has to be weighed against measured quality, latency and memory.

Part 9: conversation memory

The model "remembers" only because **we resend the whole conversation** in every request. The printed prompt shows that the last call contained `User: My name is Ahmed.` / `Assistant: Hello, Ahmed! ...` / `User: What is my name?`. The answer is simply in the input.

The LLM itself remembers nothing. Its weights are frozen and nothing is stored inside the model between two HTTP calls: each call is stateless. The "memory" is the Python list `history` in our application. (The model even says "I remember your name from our previous conversation", but that is just text it generated from the prompt.) Consequences: the history must fit in the context window (4,096 tokens here), every turn makes the prompt longer and slower, and old turns must eventually be cut or summarised — which is what `MAX_TURNS` does in the final app.

Exercise 11: structured output

Application code needs **data it can parse**, not prose. An LLM's free text can change from run to run (extra sentences, a ```` ```json ```` fence, a missing field, `"15.5"` instead of `15.5`), and `json.loads` then fails or the program reads the wrong field.

- The prompt-only approach worked here, but only by luck: nothing forces it, so in practice we need a defensive `extract_json` (strip fences, regex for `{...}`).
- With **`format=<JSON schema>`** Ollama constrains decoding so that the output **must** match the schema: all required keys are present, age is an integer, `grade` a float (`<class 'float'>`), and `risk` can only be `low` / `medium` / `high`.

This gives a **contract** between the LLM and the rest of the software: the result can be stored in a database, displayed in a UI, or used to trigger an action without fragile text parsing. It also makes testing and validation possible. (The schema guarantees the **format**, not that the content is true: the model still decides the values.)

Exercise 12: document assistant

The model respected the instruction in both cases: “What is reinforcement learning?” was answered by paraphrasing the document’s sentence (agent, environment, rewards and penalties), and “What is quantum computing?” returned exactly **“Information not available.”**, even though qwen3 certainly knows what quantum computing is from its training data. The prompt successfully restricted the model to the supplied context (**grounding**).

This is not a guarantee: with a vaguer instruction, a longer document or a smaller model, LLMs sometimes mix in their own knowledge. Making the refusal phrase explicit and repeating “only the document” helps, and the result should be tested with out-of-scope questions, as done here. In the final app, the same check (“What is quantum computing?”) also returns “Information not available.”

Part 14: local vision

- **LLM vs multimodal model:** an LLM only takes **text tokens** as input. A multimodal model (here `qwen2.5vl:7b`) also has a **vision encoder** that turns the image into a sequence of visual embeddings, projected into the same space as the text tokens. The language model then attends to both and generates text about the image.
- **Extra information received:** the **image itself**, sent base64-encoded in the `images` field of the request. The model therefore sees the pixels: it recognised the motorcycles, the brick pavement, the sunlight and shadows, and it **read the text “SUZUKI”** on the tank (OCR) — none of this was in the prompt. Its “interpretation” (a meet-up or event) is a guess, not a fact from the image.
- **Applications of local vision AI:** document analysis and **OCR** of invoices or forms, **industrial inspection** (defects on a production line), education (explaining diagrams and exercises), **medical-image pre-processing** (triage, anonymisation), surveillance and counting, accessibility (describing images for blind users), and robotics. Running it **locally** matters for exactly these cases: images are often confidential, and many of these settings are offline or need low latency.

Part 16: performance benchmark

Measured on this machine (same 500-word prompt, cold start, `think=False`):

	qwen3:8b	llama3.2:3b
Parameters / quantization	8.2 B · Q4_K_M	3.2 B · Q4_K_M
Disk size	5.23 GB	2.02 GB
Loaded size (all in VRAM)	6.03 GB	3.31 GB
Load time	1.37 s	1.04 s
Speed	173 tok/s	**286 tok/s**
Response time (≈ 680–810 tokens)	5.5 s	**4.2 s**
Quality (Ex 7)	**5/5**	2/5

Each part of the trade-off appears in the numbers. The model with **2.6× more parameters** takes **2.6× more disk** and **1.8× more memory**, loads **≈1.3× slower** and generates **1.7× fewer tokens per second**. On the other hand, it was the only one to produce correct, complete code. Both models fit easily in this GPU’s 32 GB, so speed differences come only from the amount of computation per token. On a laptop with 8 GB of RAM and no GPU, the 8B model would have to run on the CPU (several times slower), and the 3B model would be the realistic choice. A model must be chosen from measurements **on the target hardware and the target task**: for code generation here the bigger model is worth its cost; for simple chat the smaller one is enough.

Report questions

Q1. What is Local AI?

Local AI means running an AI model — here an LLM — **on your own hardware** (laptop, workstation, on-premise server) instead of calling a provider's cloud API. The weights are downloaded once, and inference (computing the answer) happens on the local CPU/GPU. In this lab, Ollama serves qwen3:8b, llama3.2:3b, qwen2.5vl:7b and nomic-embed-text on `localhost:11434`, and everything runs on the local GPU.

Q2. Difference between a Cloud LLM and a Local LLM?

	Cloud LLM	Local LLM
Where it runs	Provider's data centre	Own machine
Data	Sent over the Internet to a third party	Never leaves the machine
Internet	Required	Only to download the model
Cost	Pay per token / subscription	Hardware + electricity, no per-token cost
Model size / quality	Very large frontier models	Limited by local RAM/VRAM (mostly 1–30 B params, quantized)
Latency	Network + queue	No network; depends on local hardware
Control	Provider can change or remove the model	Fixed version, fully under our control

Q3. What role does Ollama play?

Ollama is the **local model server and manager**. It (1) **downloads and stores** models from its library (`ollama pull`, `list`, `rm`), as quantized GGUF files; (2) **loads** them into GPU/CPU memory on demand and unloads them after inactivity (`ollama ps`, `keep_alive`); (3) runs **inference** with an optimised engine (llama.cpp-based), applying each model's chat template; and (4) exposes everything through a **REST API** on `localhost:11434` (`/api/generate`, `/api/chat`, `/api/embed`, `/api/tags`, `/api/ps`…) plus a CLI. Applications therefore don't need to know anything about GPUs or weight formats; they only send HTTP requests.

Q4. What is model quantization?

Quantization stores the model's weights with **fewer bits** than the 16/32-bit floats used in training, for example 4-bit integers with a scale factor per block of weights. Our models use **Q4_K_M** (~4.5 bits per weight on average): qwen3 with 8.2 B parameters takes **5.2 GB** instead of ~16 GB in FP16. Benefits: ~3–4× less memory and disk, and faster inference, since generation is limited by memory bandwidth. Cost: a small loss of accuracy, which grows as the number of bits goes down (Q8 ≈ lossless, Q4 usually acceptable, Q2 clearly degraded). The embedding model, which is small, is kept in F16.

Q5. Why does model size affect memory consumption?

Every parameter is a number that must sit in memory during inference: **memory ≈ number of parameters × bytes per parameter**. For example 8.2 B × ~0.6 byte (Q4) ≈ 5 GB, versus 3.2 B → ≈ 2 GB. On top of the weights come the **KV cache** (it grows with context length × number of layers × hidden size, which are also larger in bigger models) and runtime buffers. That is why the loaded

sizes measured in Part 16 (6.0 GB and 3.3 GB) are larger than the files on disk (5.2 GB and 2.0 GB). If the model does not fit in VRAM, part of it moves to system RAM and generation slows down a lot.

Q6. Why can a smaller model sometimes be preferable?

Because the best model is the one that does the **job** within the **constraints**. A smaller model: fits on modest hardware (laptop, edge device, no GPU); answers **faster** (llama3.2:3b: 286 tok/s and 1.0 s load, vs 173 tok/s and 1.4 s for qwen3:8b); uses less energy; can serve more users at once on the same GPU; and is good enough for simple tasks (classification, extraction, short chat, RAG over a small document set, where the answer comes from the context anyway). It can also be **fine-tuned** cheaply for a narrow task, where it may beat a larger general model.

Q7. Advantages of local inference?

- **Privacy / confidentiality:** prompts, documents and images never leave the machine (important for medical, legal, company or student data, and for GDPR).
- **Offline operation:** works without Internet once the model is downloaded.
- **No per-token cost:** unlimited use for the price of the hardware and electricity.
- **Low and predictable latency:** no network round-trip and no provider queue or rate limits.
- **Control and reproducibility:** a fixed model version (it is not silently updated), a fixed seed, and full control of parameters and system prompts.
- **Customisation:** you can choose, quantize or fine-tune any open model.

Q8. Disadvantages of local inference?

- **Lower capability:** local models are much smaller than frontier cloud models, so reasoning, knowledge and code quality are weaker (llama3.2:3b's buggy `is_prime`).
- **Hardware requirements:** a GPU with enough VRAM is needed for good speed. On CPU only, large models are slow, and memory limits the size of the model.
- **Setup and maintenance:** installing, updating (e.g. the newest models needed a newer Ollama than the one installed here), monitoring and securing the server is our job.
- **Scalability:** one machine serves few simultaneous users. The cloud scales elastically.
- **Energy and cost up front:** you buy the hardware and pay for the electricity even when it is idle.
- **Security responsibility:** a local server exposed by mistake is unprotected (see Q15).

Q9. Explain the architecture of your Python application.

The final application (`LocalAI_Lab/source/`) is a **3-tier, all-local** architecture:

```
Browser (index.html + JS) --POST /ask (JSON)--> Flask app.py --HTTP /api/chat--
> Ollama :11434 --> qwen3:8b (GPU)
                                                          |-- rag.Retriever: TF-
IDF over documents/*.txt -> top-2 context
                                    |-- history: last MAX_TURNS=6 user/assistant pairs
                                                                          '--
STRUCT_SCHEMA: JSON schema when "structured" is ticked
```

1. The **front-end** (HTML + JavaScript) sends the question and the "structured" flag to `/ask` with `fetch`, then displays the answer with its response time, token count, tokens/s and the source documents.
2. **`app.py` (Flask)** retrieves the 2 most relevant documents with **`rag.py`**, builds the messages (system prompt + recent history + `CONTEXT` + question), calls Ollama's `/api/chat` (with `format=schema` for structured answers), measures the elapsed time and stores the turn in the history. `/reset` clears the history.

3. **Ollama** runs the model on the GPU and returns the answer plus counters (`eval_count`, `eval_duration`).

Supporting modules: `chatbot.py` (console chatbot with memory) and `benchmark.py` (model comparison). Everything listens only on 127.0.0.1.

Q10. Explain the role of the REST API.

The **REST API** is the **interface between our application and the model**. Ollama exposes HTTP endpoints that accept and return JSON (POST `/api/generate`, `/api/chat`, `/api/embed`; GET `/api/tags`, `/api/ps`). This means that: - the application does not depend on how the model is implemented: any language (Python `requests`, JavaScript, Java, curl) can use the model; - the model server and the application are **decoupled**: they can be restarted, replaced or moved to another machine by changing only a URL (`OLLAMA_URL`), and the same server is shared by the CLI, the notebook, the Flask app and the benchmark; - each request is **stateless** and self-describing (model, prompt/messages, options, format, images), and the response includes metadata useful for monitoring (durations, token counts).

Our Flask app also exposes its own small REST API (`/ask`, `/reset`) to the browser.

Q11. What is RAG?

RAG (Retrieval-Augmented Generation) combines a **retriever** with a **generator**. Instead of relying only on what the LLM memorised during training, we (1) **retrieve** the passages most relevant to the question from a document collection, (2) **insert them into the prompt** as context, and (3) ask the LLM to **answer using only that context**.

```
Question -> Retriever (keyword TF-IDF, or embeddings + cosine similarity) -> top-k passages -> Prompt -> LLM -> grounded answer
```

Benefits: up-to-date and private knowledge without retraining, fewer hallucinations, answers that can cite their sources, and an honest "Information not available" when nothing relevant is found ("Who won the 2018 World Cup?" → no document retrieved → refusal). The **quality of the retriever** limits the quality of the answer. Our keyword retriever needs shared words; with "What hardware accelerates network training?" it scored deep_learning.txt and machine_learning.txt the same (0.88), while **semantic embeddings** (nomic-embed-text) correctly ranked deep_learning.txt first (cosine 0.64 vs 0.53).

Q12. Difference between an LLM and a multimodal LLM?

A (text) **LLM** takes only **text tokens** as input and produces text. A **multimodal LLM** (here `qwen2.5vl:7b`) also accepts **other modalities**, mainly images. A **vision encoder** (a ViT) turns the image into patch embeddings, a **projector** maps them into the LLM's token-embedding space, and these "visual tokens" are placed in the sequence next to the text tokens, so the language model attends to both. It can therefore describe scenes, read text (it read "SUZUKI" on the motorcycle), answer questions about an image, and analyse documents or charts. The cost is a larger model (6.0 GB vs 5.2 GB here) and more tokens per request (one image becomes hundreds of tokens).

Q13. Compare two local models experimentally.

Same machine, same prompts, Ollama 0.12.6, 100 % GPU:

Metric	qwen3:8b	llama3.2:3b
Parameters / quant.	8.2 B · Q4_K_M	3.2 B · Q4_K_M
Disk / loaded size	5.23 GB / 6.03 GB	2.02 GB / 3.31 GB
Load time (cold)	1.37 s	1.04 s

Metric	qwen3:8b	llama3.2:3b
Generation speed	173 tok/s	286 tok/s
500-word answer	5.5 s	4.2 s
Prime function (Ex 7)	18.7 s with thinking · correct	2.1 s · **bug for n = 5**, no complexity
Quality / correctness (Ex 7)	5/5 · 5/5	2/5 · 3/5

The small model is **≈1.7× faster** and uses **≈half the memory**, but the large one gives **clearly better, correct and complete** answers. qwen3 is also a "thinking" model: by default it generates hidden reasoning tokens (3,065 tokens for the prime question), which improves quality but increases latency. It can be disabled with `think=False` when speed matters (JSON, RAG and benchmark cells).

Q14. Which model would you deploy on a computer with limited RAM? Explain.

On a computer with limited RAM (e.g. 8 GB, no dedicated GPU) I would deploy the **small model, llama3.2:3b (Q4_K_M)**. It needs only **≈2 GB on disk and ≈3.3 GB in memory**, which leaves room for the operating system, the browser and the application. qwen3:8b needs ≈6 GB, which is too tight and would cause swapping. It also loads faster and generates more tokens per second, which matters even more on a CPU. Its weaknesses (complex code, reasoning) can be reduced by using it for what small models do well — short answers, extraction, and **RAG**, where the facts come from the retrieved documents — and by validating outputs (JSON schema, tests). If the RAM were even smaller, I would go down to a 1B model or a more aggressive quantization (Q3), while checking quality on the real task.

Q15. What security risks exist when exposing a local LLM server to a network?

Answer. By default Ollama listens on 127.0.0.1 (verified here: `127.0.0.1:11434`) and has **no authentication, no encryption and no rate limiting**. If it is exposed to a network (`OLLAMA_HOST=0.0.0.0`, port forwarding, a cloud VM with an open port), then: - **Unauthorised use of compute:** anyone who can reach the port can run unlimited generations on our GPU — free AI for them, cost, energy and denial of service for us. - **Model management by strangers:** the API can also **pull, create, copy and delete** models (`/api/pull`, `/api/delete`): they could fill the disk, delete our models or replace one with a modified one. - **Data exposure:** prompts and answers travel **in clear text** (HTTP), so they can be sniffed on the network. Through our own app, an attacker could also query our private **RAG documents**. - **Prompt injection:** in a RAG system, any text in the documents is read by the model as instructions ("ignore previous instructions and …"). A malicious document or web page could make the assistant leak data, or call tools if the app gave it any. - **Vulnerabilities in the server itself:** an exposed service is attacked directly, and there have been public CVEs in Ollama (e.g. path traversal and DoS issues), so the version must be kept updated. - **Our Flask app:** `debug=True` would give a remote Python console. We use `debug=False` and bind to `127.0.0.1`, and the history is shared by all users.

Mitigations: keep Ollama on localhost; put a **reverse proxy** (nginx/Caddy) in front with **authentication** (API key, OAuth) and **TLS (HTTPS)**, plus rate limits; **firewall** the port (allow only trusted IPs, or use a VPN/SSH tunnel); never run Flask in debug mode in production; treat retrieved documents as untrusted data; and keep Ollama updated.